Practice 2 - Stretch Robot Kinematics

Robotics and Cyber-Physical Systems
School of Sciences and Engineering
Tecnológico de Monterrey

Introduction

This practice introduces the fundamental concepts of robot control through six progressive parts. You will learn how to command joint velocities, read sensor state, implement position control, process input devices, layer commands with proportional blending, and integrate full teleoperation with autonomous behaviors.

Each part builds on the previous one, developing a complete understanding of how to control the Stretch robot through the `stretch_toolkit` interface.

Before You Begin

If you previously cloned this repository, update it to the latest version to ensure you have all the necessary files and recent improvements. **Important:** The update command below will overwrite any local changes you've made. If you have files you want to keep (such as solutions to previous activities), make a backup copy outside the repository folder before proceeding.

To update your repository:

```
git pull
git fetch origin
git reset --hard origin/main
```

If dependencies were updated, synchronize your environment:

```
uv sync
```

1 Part 1: Joint Velocity Control

The primary way to move the robot is through `controller.set_velocities()`, which accepts a dictionary mapping joint names to normalized velocities in the range $[-1.0,\ 1.0]$:

```
controller.set_velocities({
    "lift_up": 0.5,                # lift at 50% speed upward
    "arm_out": -1.0,               # arm retracting at full speed
    "head_pan_counterclockwise": 0.3,
})
```

Joint names follow a directional convention: positive values move in the named direction, negative values move in the opposite direction. Below is the full set of controllable joints:

Joint name	Positive direction
`base_forward`	Translate forward
`base_counterclockwise`	Rotate left
`lift_up`	Lift upward
`arm_out`	Arm extend
`gripper_open`	Open gripper
`wrist_yaw_counterclockwise`	Wrist yaw left
`wrist_pitch_up`	Wrist pitch up
`wrist_roll_counterclockwise`	Wrist roll left
`head_pan_counterclockwise`	Head pan left
`head_tilt_up`	Head tilt up

Activity

Starting from `stretch_boilerplate.py`, make the robot's head pan joint oscillate back and forth using a sine wave. The velocity sent to the joint at each tick should be:

$$v(t) = \sin(\omega \cdot t), \quad \omega = 2.0$$

where t is the elapsed time in seconds and ω controls the oscillation speed.

Steps:

1. Copy `stretch_boilerplate.py` and rename it `p2_1_velocity.py`.
2. Add `import math` at the top.
3. Inside the loop, compute the velocity as $v(t) = \sin(\omega \cdot t)$ with $\omega = 2.0$, where `t` is already provided by the boilerplate as `controller.get_time()`.
4. Pass it to `controller.set_velocities()` for the `head_pan_counterclockwise` joint.
5. Run with `uv run p2_1_velocity.py` and observe the head panning.
6. Experiment with different values of ω and observe the effect on oscillation speed.

Expected result: The robot's head pans smoothly left and right in a continuous sinusoidal motion.

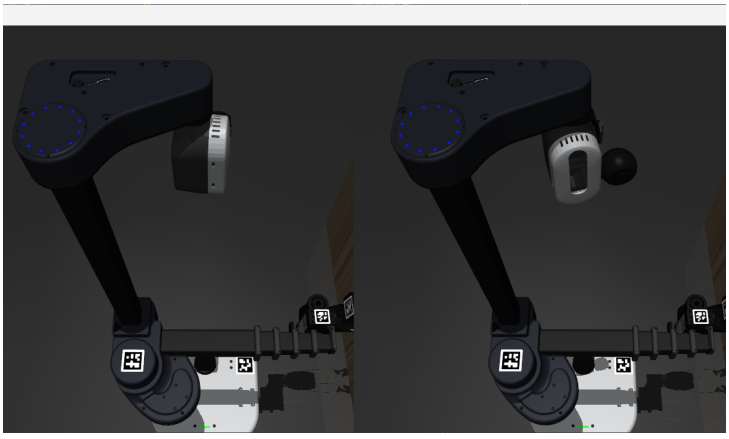


Figure 1: Robot head panning back and forth using sinusoidal velocity control.

2 Part 2: Reading Joint State

In addition to sending commands, you can read the current position of every joint using `controller.get_state()`, which returns a dictionary of joint positions:

```
state = controller.get_state()
print(state["head_pan_counterclockwise"])  # current angle in radians
```

The keys match the joint names from Part 1. Linear joints (`lift_up`, `arm_out`) are in meters; all rotational joints are in radians. The base keys are odometry estimates and do not follow the directional naming convention:

Key	Unit	Description
`base_x`	meters	Base position along X axis (odometry)
`base_y`	meters	Base position along Y axis (odometry)
`base_theta`	radians	Base heading (odometry)
`lift_up`	meters	Lift height
`arm_out`	meters	Arm extension
`wrist_yaw_counterclockwise`	radians	Wrist yaw angle
`wrist_pitch_up`	radians	Wrist pitch angle
`wrist_roll_counterclockwise`	radians	Wrist roll angle
`head_pan_counterclockwise`	radians	Head pan angle
`head_tilt_up`	radians	Head tilt angle
`gripper_open`	radians	Gripper opening angle

Activity

Starting from your completed `p2_1_velocity.py`, add a live printout of the head pan joint position.

Steps:

1. Copy `p2_1_velocity.py` and rename it `p2_2_state.py`.
2. Inside the loop (after setting velocities), call `controller.get_state()` and store the result.
3. Print the value of the `head_pan_counterclockwise` key on each tick.
4. Run with `uv run p2_2_state.py` and observe the printed values change as the head pans.

Expected result: The terminal prints a stream of angle values (in radians) that oscillate as the head pans left and right.

```
-2.7064544902869696
-2.7912609565602504
-2.8279074389854637
-2.8050669215428670
-2.7038859728338150
-2.6026917811471906
-2.4689193011507530
...
```

3 Part 3: Position Control

Sending raw velocities works well for continuous motion, but reaching a *specific* position requires knowing when to stop. `StateController` handles this: given a desired joint position, it computes the velocity needed to move toward it each tick, and reports when the target has been reached.

Crucially, `StateController` does not bypass the velocity interface — it simply *generates* the velocity commands for you. The robot is still driven through `controller.set_velocities()`, keeping a single point of control over all motion.

`StateController` uses the same joint names as `set_velocities()` and `get_state()`.

Basic usage pattern:

```
from stretch_toolkit import StateController

sc = StateController(controller, {"arm_out": 0.3})  # target: extend arm to 0.3 m

while True:
    controller.set_velocities(sc.get_command())   # drive toward target
    if sc.is_at_goal():
        break                                      # stop when reached
```

Optionally, you can track how far along a movement the robot is by calling `get_progress(previous_state)`, passing the state captured *before* the movement began. It returns a dictionary mapping each joint to a value between 0.0 (not started) and 1.0 (arrived). This is not required for the activity, but can be useful for certain applications.

```
initial = sc.get_current_state()

while not sc.is_at_goal():
    controller.set_velocities(sc.get_command())
    progress = sc.get_progress(initial)
    print(f"arm progress: {progress['arm_out']:.2f}")
```

Activity

Make the arm joint bounce continuously between 0.1 m and 0.3 m.

Steps:

1. Copy `stretch_boilerplate.py` and rename it `p2_3_position.py`.
2. Import `StateController` from `stretch_toolkit`.
3. Before the loop, create a variable `target = 0.1` and create a `StateController` with `{"arm_out": target}`.
4. Inside the loop, call `sc.get_command()` and pass the result to `controller.set_velocities()`.
5. When `sc.is_at_goal()` is `True`, use an if/else statement to toggle `target`: if it equals 0.1, set it to 0.3; otherwise set it to 0.1. Then create a new `StateController` with the updated target.
6. Run with `uv run p2_3_position.py` and watch the arm extend and retract.

Template for the toggle logic:

```
if sc.is_at_goal():
    if target == 0.1:
        target = 0.3
```

```
    else:
        target = 0.1
    sc = StateController(controller, {"arm_out": target})
```

Expected result: The arm smoothly travels from 0.1 m to 0.3 m and back, repeating indefinitely.

Important: Make sure this file works correctly before moving on. Parts 5 and 6 will build directly on this code, so it's essential that you have a working `p2_3_position.py` that bounces the arm reliably. Test it for at least a few complete cycles to verify the behavior.

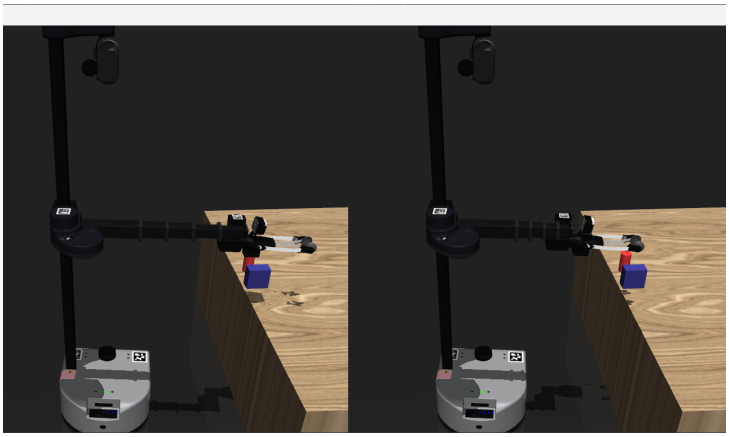


Figure 2: Robot arm extending and retracting using position control.

4 Part 4: Gamepad and Keyboard Input

The `stretch_toolkit.input` module provides a unified interface for reading gamepad buttons, analog axes, and keyboard keys. Import it with:

```
import stretch_toolkit.input as inp
```

Buttons and keys: `is_pressed`

`inp.is_pressed(name)` returns `True` as long as the given button or key is held down. It checks the gamepad first, then falls back to the keyboard.

```
if inp.is_pressed("A"):          # gamepad A button
    ...
if inp.is_pressed("w"):          # keyboard 'w' key
    ...
```

The full set of gamepad button names is:

Name	Button
`A`, `B`, `X`, `Y`	Face buttons
`LB`, `RB`	Bumpers
`LT`, `RT`	Triggers (also available as axes)
`L3`, `R3`	Stick clicks
`DPAD_UP`, `DPAD_DOWN`, `DPAD_LEFT`, `DPAD_RIGHT`	D-pad
`START`, `SLCT`, `GUIDE`	Menu buttons

Analog axes: `get_axis`

`inp.get_axis(name)` returns a normalized float. Sticks return $[-1.0,\ 1.0]$; triggers return $[0.0,\ 1.0]$.

```
lx = inp.get_axis("LX")    # left stick horizontal
rt = inp.get_axis("RT")    # right trigger (0.0 to 1.0)
```

Available axes:

Name	Source	Range
`LX`, `LY`	Left stick	$[-1.0,\ 1.0]$
`RX`, `RY`	Right stick	$[-1.0,\ 1.0]$
`LT`, `RT`	Triggers	$[0.0,\ 1.0]$
`DPAD_X`, `DPAD_Y`	D-pad	$\{-1,\ 0,\ 1\}$

Single-frame detection: `rising_edge`

`inp.rising_edge(name)` returns `True` exactly once — on the first frame a button goes down. Subsequent calls return `False` until the button is released and pressed again. This is useful for toggling state or triggering a one-shot action:

```
if inp.rising_edge("A"):
    print("A was just pressed")    # fires once per press
```

5 Part 5: Command Layering

In practice, you often want to run an autonomous behavior while still being able to override it manually. The challenge is combining two command dictionaries — one from a controller or algorithm, one from the gamepad — without simply discarding one of them.

`merge_proportional` solves this. It takes a *primary* command (typically teleop) and a *secondary* command (typically autonomous), and blends them based on how much the primary input is active:

```
from stretch_toolkit import merge_proportional

merged = merge_proportional(cmd_primary, cmd_secondary)
controller.set_velocities(merged)
```

The blending rule per joint is:

- If the primary input magnitude is below a deadband (default 0.05), the secondary command is used unchanged.
- Otherwise, the primary input strength (0 to 1) determines how much the secondary is overridden: at full deflection the joint moves at full speed in the primary direction; at partial deflection the output is a weighted blend.

This means a light touch on the stick nudges the robot away from its autonomous target, while a full push takes over completely. Releasing the stick hands control back to the secondary command seamlessly.

A typical pattern combining autonomous position control with manual override looks like:

```
from stretch_toolkit import controller, StateController,
    merge_proportional
import stretch_toolkit.input as inp
```

```
3
4  sc = StateController(controller, {"arm_out": 0.3})
5
6  while True:
7      cmd_auto   = sc.get_command()
8      cmd_teleop = {"arm_out": inp.get_axis("RX")}
9      controller.set_velocities(merge_proportional(cmd_teleop, cmd_auto))
```

Here, moving the right stick overrides the autonomous arm motion proportionally; leaving the stick centered lets the `StateController` drive undisturbed.

Activity

Starting from your completed `p2_3_position.py`, add a manual override that extends the arm fully while the `'v'` key is held.

Steps:

1. Copy `p2_3_position.py` and rename it `p2_5_layering.py`.
2. Inside the loop, build a primary command dict: set `"arm_out"` to `1.0` if `inp.is_pressed("v")` is `True`, otherwise `0.0`.
3. Use `merge_proportional` to merge the primary command with `sc.get_command()`, and pass the result to `controller.set_velocities()`.
4. Run with `uv run p2_5_layering.py`. The arm should bounce as before. While holding `'v'`, it should override and extend fully.

Expected result: The arm bounces between 0.1 m and 0.3 m autonomously, but pressing `'v'` overrides the motion and drives the arm toward full extension.



Figure 3: Manual override taking priority over autonomous position control.

6 Part 6: Full Teleop Override

In Part 5, you built a manual override by constructing a primary command dict by hand. In practice, you want the operator to be able to override *any* joint at any time using the full gamepad and keyboard mapping — not just a single hard-coded key.

`TeleopProvider` already knows how to read all joints from the input devices. Its `get_manual_override(cmd_` method combines this with `merge_proportional` in a single call: it reads the current teleop input, uses it as the primary command, and blends it with the autonomous command you provide.

```
1  from stretch_toolkit import controller, teleop, StateController
2
3  sc = StateController(controller, {"arm_out": 0.3})
4
```

```
while True:
    controller.set_velocities(teleop.get_manual_override(sc.get_command
        ()))
```

This replaces both `teleop.get_normalized_velocities()` and the manual `merge_proportional` call with a single line, while giving the operator proportional control over every joint simultaneously.

Additionally, pressing the `Y` key (or `y`) toggles between two modes:

- **AUTONOMOUS mode** (default): Proportional blending — the operator can override any joint, but the autonomous command runs when inputs are idle.
- **MANUAL mode**: Pure teleop control — the autonomous command is ignored completely, and only the operator's inputs drive the robot.

Activity

Starting from your completed `p2_3_position.py`, replace the raw `set_velocities` call with a full teleop override.

Steps:

1. Copy `p2_3_position.py` and rename it `p2_6_teleop_override.py`.
2. Import `teleop` from `stretch_toolkit`.
3. Replace `controller.set_velocities(sc.get_command())` with `controller.set_velocities(teleop.g`
4. Run with `uv run p2_6_teleop_override.py`. The arm bounces as before, but now any mapped key or gamepad input can override any joint proportionally.
5. Press `Y` (or `y`) to toggle to MANUAL mode. The arm stops bouncing and responds only to your inputs. Press `Y` again to return to AUTONOMOUS mode.

Expected result: In AUTONOMOUS mode, the arm bounces while you can proportionally override any joint. In MANUAL mode, the arm stops bouncing and responds only to your direct teleop control. The toggle gives you instant control over whether autonomous behavior runs or not.

Deliverable

Submit a video demonstrating your completed `p2_6_teleop_override.py` program. The video should show:

1. The robot arm bouncing autonomously between 0.1 m and 0.3 m in AUTONOMOUS mode.
2. You using gamepad or keyboard inputs to proportionally override the arm's motion while it continues bouncing.
3. You pressing the `Y` key to toggle to MANUAL mode (the arm should stop bouncing and respond only to your inputs).
4. You controlling the robot manually in MANUAL mode.
5. You pressing `Y` again to return to AUTONOMOUS mode (the arm should resume bouncing).

The video should be clear enough to demonstrate that both modes work correctly and that the proportional override functions as expected. Keep the video concise — 30 to 60 seconds is sufficient.